

Informed Search: Pattern Database

Heuristics Part-7

Introduction

In the pursuit of efficient search algorithms, particularly for complex problems like the 15-puzzle or Rubik's Cube, the choice of heuristic function plays a pivotal role. While a more "informative" heuristic can dramatically reduce the number of nodes explored during a search, it often comes at the cost of increased computational expense for the heuristic itself. This presents a fundamental trade-off: how much effort should be invested in computing a heuristic versus the effort saved in the actual search process? Understanding this balance is crucial for designing practical and performant AI systems.

This lecture delves into this critical phenomenon, exploring the relationship between heuristic informativeness, heuristic computation cost, and the overall search time. We will examine the limitations of simpler heuristics and then introduce Pattern Database (PDB) heuristics, a powerful technique that pre-computes highly informative heuristic values, effectively shifting the computational burden offline to achieve significant speedups during online search. This approach challenges conventional wisdom regarding heuristic complexity and opens new avenues for solving previously intractable problems.

Core Concepts

Term	Definition	Significance
Heuristic Informativeness	A measure of how closely a heuristic function estimates the true cost to reach the goal state.	More informative heuristics lead to fewer nodes being explored by search algorithms like A or IDA.
Heuristic Computation Cost	The computational resources (e.g., time) required to calculate the heuristic value for a given state.	This cost adds to the total search time and must be balanced against the benefits of a more informed heuristic.
Search Cost		

Term	Definition	Significance
	The computational resources (e.g., time, memory) expended by the search algorithm to find a solution, primarily due to node expansions.	A primary goal of informed search is to minimize this cost by guiding the search effectively.
Total Cost	The sum of the heuristic computation cost and the search cost.	The ultimate metric for evaluating the efficiency of a search strategy, aiming for a minimum total cost.
Admissible Heuristic	A heuristic function that never overestimates the true cost to reach the goal state.	Guarantees that algorithms like A* will find an optimal solution.
Linear Conflict	A situation in sliding tile puzzles where two tiles are in their goal row/column but in reversed order, requiring extra moves to resolve.	Highlights a limitation of simple heuristics like Manhattan distance, which fail to account for such interactions.
Domain Relaxation	Simplifying the original problem by removing certain constraints or interactions to make it easier to solve.	A common technique for deriving admissible heuristics, as the optimal solution to the relaxed problem provides a lower bound.
Pattern Database (PDB)	A pre-computed lookup table storing the exact number of moves required to solve a specific subset of tiles (a "pattern") to their goal positions, ignoring other tiles.	Allows for highly accurate and admissible heuristic values to be retrieved quickly during search, overcoming online computation costs.
Additive Pattern Databases	A method of combining multiple PDBs where the patterns are disjoint (no tile belongs to more	Provides a way to create more informed heuristics by combining

Term	Definition	Significance
	than one pattern), allowing their heuristic values to be summed for a more powerful admissible heuristic.	independent sub-problem solutions without double-counting moves.

Detailed Analysis

1. The Heuristic Cost vs. Search Cost Trade-off

The efficiency of an informed search algorithm is a delicate balance between the cost of computing the heuristic and the cost of the search itself. Intuitively, a more informative heuristic (closer to the optimal h^*) will reduce the number of nodes expanded during search, thus lowering the search cost. Conversely, computing a highly informative heuristic often requires more computational effort.

Consider a graph where the x-axis represents heuristic informativeness (from h_0 - no information, to h^* - perfect information) and the y-axis represents the cost of computing the solution (running time).

- **Heuristic Computation Cost:** As informativeness increases, the cost to compute the heuristic generally rises, potentially exponentially, because more complex calculations are involved (e.g., counting misplaced tiles vs. calculating shortest paths in a relaxed problem).

- **Search Cost:** A more informative heuristic leads to a significant reduction in the number of nodes expanded, thus decreasing the search cost. For instance, in the 15-puzzle, A with h_1 (misplaced tiles) might expand 39,000 nodes, while A with h_2 (Manhattan distance) might expand only 6,000 nodes.

- **Total Cost:** The total cost is the sum of heuristic computation cost and search cost. The optimal heuristic lies at the minimum point of this total cost curve.

Historically, the conventional wisdom among search researchers was that the global minimum for total cost often occurred with cheaper, lower-order polynomial heuristics. The reasoning was that if a heuristic itself was exponentially expensive to compute, it would negate any search savings. However, recent advancements, particularly in the last 10-15 years, have challenged this view, demonstrating that sometimes more expensive heuristics, like Pattern Databases, can lead to overall better performance.

2. Limitations of Traditional Heuristics (e.g., Manhattan Distance)

While heuristics like Manhattan distance (h_2) significantly improve search performance over simpler ones like misplaced tiles (h_1), they still have limitations,

especially for larger problems.

- **Example: 15-puzzle:** IDA with Manhattan distance can solve a random 15-puzzle instance in about 50 seconds, generating around 400 million nodes. This is a vast improvement over brute force (which might take 6 days) but still not instantaneous.
- **Example: 24-puzzle:** For the 24-puzzle, IDA with Manhattan distance would take an average of 65,000 years. This highlights that even a good heuristic can be insufficient for larger problem instances.

The core problem with Manhattan distance is its assumption of tile independence. It calculates the sum of individual tile distances to their goal positions, ignoring interactions between tiles.

- **Linear Conflict:** Consider two tiles, say '3' and '1', that need to swap positions in a row (e.g., '3' at (1,3) needs to go to (1,1), and '1' at (1,1) needs to go to (1,3)). Manhattan distance would calculate their combined contribution as 4 (2 moves for '3' + 2 moves for '1'). However, due to the linear conflict, one tile must move out of the way for the other, requiring at least 6 moves. Manhattan distance underestimates the true cost because it doesn't model these necessary interactions.
- **Underestimation:** This underestimation can be substantial. For a set of red tiles, Manhattan distance might estimate 19 moves, while 31 are actually needed. For another configuration, it might estimate 20 moves when 28 are needed, or 17 when 27 are needed. These discrepancies lead to more node expansions and longer search times.

Introduction

Pattern Database (PDB) heuristics address the limitations of simpler heuristics by incorporating more complex tile interactions without incurring high online computation costs. The core idea is based on **domain relaxation**:

1. **Relaxation:** Instead of considering all tiles, we select a subset of "pattern tiles" (e.g., the red tiles in an example). All other tiles (the "white" tiles) are treated as indistinguishable or "don't care" tiles. The problem is relaxed by allowing white tiles to move freely without affecting the pattern tiles' path cost, or by simply ignoring their final positions.
2. **Admissibility:** The optimal cost to move the pattern tiles to their goal positions in this relaxed problem is an admissible heuristic for the original problem, as it's a lower bound on the true cost.
3. **Pre-computation:** Solving this relaxed problem for every possible configuration of the pattern tiles can be computationally expensive. However, this expensive operation is performed once, offline, before any actual search begins.
4. **Database Storage:** The computed heuristic values for all pattern tile configurations are stored in a lookup table, or "pattern database." This database is indexed for quick retrieval.

5. **Online Retrieval:** During the actual search, when a heuristic value is needed for a state, the algorithm simply queries the pre-computed database, retrieving the value instantly. This avoids the high online computation cost of complex heuristics.

This approach effectively shifts the computational burden from online search to an offline pre-computation phase, allowing for the use of highly informed heuristics without slowing down the search itself.

4. Constructing and Using Pattern Databases

A pattern database is a complete set of states for a chosen pattern of tiles, along with the associated minimum number of moves to reach their goal configuration.

- **Backward Search:** PDBs are typically constructed using a breadth-first search (BFS) performed backward from the goal state of the relaxed problem. Starting from the goal configuration of the pattern tiles, BFS explores all reachable states, recording the shortest path length (number of moves) to the goal for each state. This ensures that the stored values are optimal for the relaxed problem and thus admissible.

- This is analogous to A search, but in reverse. The lecture also mentions that bi-directional A is a related advanced topic.

- **Indistinguishable Tiles:** During PDB construction, the non-pattern tiles (e.g., "white" tiles) are treated as indistinguishable. Their specific identities are ignored, but their presence and movement capabilities are still considered to ensure the moves counted for the pattern tiles are valid. The PDB counts the moves of the pattern tiles, allowing non-pattern tiles to move as needed without counting their specific moves towards the heuristic.

- **Size of PDBs:** PDBs can be very large. For example, a 7-tile pattern database for the 15-puzzle might contain 519 million entries. These are computed in batches to optimize the process.

5. Combining Pattern Databases: Max vs. Additive

To create even more powerful heuristics, multiple pattern databases can be combined.

- **Max Heuristic:** If two PDBs (e.g., one for red tiles, one for blue tiles) are created where the tile sets might overlap or their moves are not independent, the combined admissible heuristic is the maximum of the individual PDB values. For example, if a red PDB suggests 31 moves and a blue PDB suggests 22 moves, the combined heuristic would be 31. Summing them would lead to double-counting moves and thus an inadmissible heuristic.

- **Additive Pattern Databases:** A more powerful combination is possible if the pattern tile sets are **disjoint** (no tile belongs to more than one pattern) and their

moves are counted independently. In this case, the heuristic values from multiple PDBs can be summed to produce a more accurate and still admissible heuristic.

- For example, if we divide the 15-puzzle into two disjoint sets of tiles (e.g., tiles 1-7 and tiles 8-15), we can create a PDB for each set. If the first PDB (e.g., for tiles 1-7) indicates 20 moves are needed, and the second PDB (e.g., for tiles 8-15) indicates 25 moves are needed, the total additive heuristic would be 45. This is admissible because the moves for each set of tiles are counted independently, without double-counting.
- Manhattan distance can be seen as a special case of additive PDBs where each pattern contains only a single tile.

6. Performance Improvements with Pattern Databases

The impact of PDBs on search performance is dramatic:

- **15-puzzle:** IDA using two additive PDBs (e.g., 7-tile and 8-tile patterns) can solve the 15-puzzle optimally in approximately 30 milliseconds, representing a 2000x speedup compared to Manhattan distance.
- **24-puzzle:** For the 24-puzzle, which previously required 65,000 years with Manhattan distance, additive PDBs can solve a random instance in about 2 days. While still long, this is a monumental improvement, making the problem tractable. Modern implementations, building on 15-year-old research, likely achieve even faster solutions.
- **Complex PDBs for 24-puzzle*:** To achieve these results for the 24-puzzle, researchers often use multiple additive PDBs, for example, four 6-tile databases, carefully designed to cover the entire set of tiles without overlap.

Key Takeaways

- [] The total cost of solving a problem involves a trade-off between heuristic computation cost and search cost.
- [] More informative heuristics generally reduce search cost but increase heuristic computation cost.
- [] Simple heuristics like Manhattan distance can be insufficient for larger problems due to their inability to model complex tile interactions (e.g., linear conflicts).
- [] Pattern Database (PDB) heuristics pre-compute optimal solutions for relaxed sub-problems, storing them in a lookup table.
- [] PDBs allow for highly accurate and admissible heuristic values to be retrieved quickly during online search, shifting computational burden offline.
- [] PDBs are typically constructed using backward breadth-first search from the goal state of the relaxed problem.

- [] When combining multiple PDBs, if tile patterns can overlap or interact, the maximum of their values must be taken to maintain admissibility.
- [] If pattern tile sets are disjoint, their individual PDB values can be summed to create a more powerful and still admissible additive heuristic.
- [] Additive PDBs provide significant speedups, making previously intractable problems like the 24-puzzle solvable within reasonable timeframes.

Conclusion

The exploration of Pattern Database heuristics marks a significant advancement in informed search, demonstrating how a strategic investment in offline computation can yield profound benefits during online problem-solving. By meticulously pre-computing optimal solutions for relaxed sub-problems and storing them in accessible databases, PDBs effectively overcome the limitations of simpler heuristics that fail to capture complex interactions. This approach has transformed the landscape of solving classic AI search problems, making puzzles like the 15-puzzle and even the daunting 24-puzzle tractable within practical timeframes.

The insights gained from PDBs extend beyond puzzles, influencing the design of heuristics for various combinatorial search problems. They highlight the power of domain relaxation and the importance of carefully balancing the cost of heuristic generation with the efficiency of the search process. As we move forward, understanding these advanced heuristic techniques provides a foundation for tackling even more complex challenges, paving the way for the next generation of intelligent problem-solving agents. The principles of pre-computation and intelligent decomposition, as exemplified by PDBs, will continue to be vital tools in the AI practitioner's arsenal.